

# SynapNote 营销系统 - 面试亮点讲解话术

---

## 📄 项目总体介绍

"在这段实习中，我主要负责了 **SynapNote AI 会议助手的营销落地页与运营数据系统** 的前端开发。

这个系统为了支撑产品的早期出海推广和用户获取，主要分成了前、后台两部分。

**前台**是对外的营销落地页。当用户点击我们在 Facebook、Instagram 等渠道投放的广告时，就会跳转到这个落地页。它主要负责向用户展示产品的 AI 功能与特性，并收集他们的邮箱和反馈建议。

**后台**则是对内的运营数据看板。它将落地页收集到的全量数据——包括用户的邮箱、反馈信息以及追踪广告来源的 UTM 参数——进行了统一的展示。在这块看板上，运营同学不仅可以按不同条件进行自由筛选，还能看到前端实时计算呈现的分层线索和CVR 转化率指标。

通过这两端的开发和串联，我帮团队打通了从'流量获取到转化归因'的完整链路。"

## ✨ 亮点一：PC / Mobile 双端组件独立渲染与精细化适配

回答：

"当时我们开发这个出海营销落地页时，老板明确要求 PC 端和移动端必须有两套各自适应的 UI 体验，用来做邮箱和反馈收集等转化入口。所以当时 UI 设计师直接给了我两套差异比较大的figma原型图。如果用 CSS 媒体查询的方法去写在一套结构里，比较难还原两端的布局，还会导致一套 HTML 里掺杂大量的 `display: none`。

**性能问题：**移动端用户会被迫下载大量 PC 端专属的 CSS 和交互脚本，首屏加载会非常吃亏。对于一个花钱买流量的投放页面来说，首屏慢 1 秒，跳出率就会急剧上升，直接影响转化。

**所以就没有采用媒体查询的方式，直接从组件级进行物理隔离，给 PC 和移动端各自设计一个完全独立的组件去实现。**

具体做法是，我编写了一个 `detectDevice` 函数，在页面初始化时利用 `navigator.userAgent` 获取用户设备信息，并判断设备是否是移动端。然后在 Vue 的 `App.vue` 入口里，定义一个响应式的 `isMobile` 接收 `detectDevice` 函数的返回值，再通过 `v-if / v-else` 判断一下是渲染PC端还是移动端。

在移动端的具体适配上，我使用了 **PostCSS-px-to-viewport** 方案将样式转换为 `vw` 来实现像素级还原。**为什么选择 VW 方案呢？** 主要是为了性能。相比于传统 `rem` 方案需要依赖内联 JS 去动态计算视口并监听 `resize` 事件，`vw` 的转换是在 Vite 的构建阶段就提前完成的，这极大减轻了客户端浏览器的渲染压力。同时，借助刚才提到的组件级拆分，我在 PostCSS 里配置了 `selectorBlackList` 黑名单，把 PC 端组件的类名全都过滤排除了，这样它就只会转换移动端的样式，避免了两端的响应式干扰。"

---

## ✨ 亮点二：打通“流量来源-用户行为-转化归因”全链路数据闭环

回答：

(辅助理解的概念备忘)

- **UTM 参数** (Urgency Tracking Module) 是附加在 URL (网址) 末尾的一段特定代码。它的作用是帮助网站所有者追踪流量来源。在代码里, 我们使用原生 API `window.location.search` 获取 URL 后的字符串, 然后借由 `new URLSearchParams()` 将字符串转换为对象, 方便提取。
- **PostHog** 是一个一站式分析平台, 可以追踪页面浏览量、点击次数、自定义事件。尤其是它的**录屏回放功能**, 可以记录用户在网站上的鼠标移动、点击、滚动等真实操作过程; 还有它的**\*\*可视化漏斗分析 (Funnel) \*\***功能, 能看清用户在哪一步流失, 还能统计停留时间和访客地区等信息。
- **SDK** 全称是 Software Development Kit (软件开发工具包)。

整个核心数据的串联流程是这样的:

1. **首先是解析与身份绑定:** 当用户点击特定广告进入落地页时, URL 会自动携带 UTM 参数 (如广告来源 Facebook 或 Instagram、广告组别等)。我在前端封装了一个解析模块, 把提取的 UTM 字符串解析为对象并马上发给后端, 换取一个唯一对应的 `Session ID`。
2. **其次是状态保持:** 考虑到用户操作时可能会刷新页面或中途去别的网页, 我将这个 `Session ID` 持久化存储到了 `localStorage` 里, 防止追踪断联。
3. **最后是转化携带与多端上报:** 当用户最终触发表单提交时, 前端在发送请求时不仅会把业务表单数据传给后端, 还会携带这个 `Session ID` 给后端, 这就确保了转化数据来源的准确和一致性。

在表单提交成功的同时, 我还结合了第三方埋点。具体做法是, 我封装了一个 `composable` 函数 `usePostHog`, 在函数内部通过 PostHog 网站的账户 API Key 完成 SDK 初始化, 然后将 `posthog` 实例导出。落地页组件引入这个实例后, 就可以直接调用 `capture` 记录用户行为事件、调用 `identify` 绑定用户身份, 方便后续在 PostHog 看板上分析用户的转化漏斗和行为数据。

---

## 🌟 亮点三: 搭建纯前端响应式 CVR 实时计算与 A/B 测试看板

回答:

“ROI 的全称是 Return on Investment, 中文翻译为‘投资回报率’。

这个运营决策看板, 实现的功能首先是展示落地页页面传回的数据, 也就是 `sessionid`、用户填写的邮箱, 反馈内容, 和 UTM 参数。运营同学可以在看板上自由组合筛选条件, 比如按设备类型、广告来源、广告组别或者具体的时间段来拉取数据。筛选完之后前端通过 `computed` 实时计算当前的 CVR 转化率, 同时通过 `computed` 和 `filter` 计算分层线索比如仅 UTM 的数量, 仅填邮箱的数量, 仅填写反馈的数量等。

然后的话是支持基于 UTM 的 A/B 测试。通过看板的筛选器, 运营同学可以在这里进行控制变量法的比对。比如, 控制投放渠道不变, 单对比‘Facebook AI 自动投放’和‘手动精准投放’这两种方式, 到底哪种方法收集到的有效线索更多、ROI 效益更好。

通过这个看板, 团队在跑各类 A/B 测试时终于有了**可量化的实时数据支撑**。他们能根据数据表现迅速去迭代更吸引人的广告内容, 或是选择更高效的投放方式, 最终大幅优化整体的投放 ROI。”

**补充:** 在这块看板的开发中, 关于多维度的漏斗筛选和 CVR 计算, 在 MVP (最小可行性产品) 阶段, 后端没有去写去复杂的 SQL 聚合接口, 而是把原始明细数据拉到前端, **纯粹利用 Vue 3 的 `computed` 响应式依赖收集机制在前端即时重算**。因为项目早期数据量比较少, 前端完全能够处理, 还能根据运营的反馈实时修改。